

Especificación de tokens y expresiones regulares

Compiladores - Equipo 07
Yañez Barajas Brandon - Model Designer
2 de octubre de 2026

1. Alcance

Tomé como base las categorías trabajadas en clase. El lexer leerá una cadena o archivo de izquierda a derecha y registrará categoría, lexema, línea y columna. Los espacios, tabulaciones y saltos de línea se omiten del conteo, pero actualizan la posición. Esta propuesta usa expresiones regulares en Python, sin FLEX.

2. Patrones definidos

Los patrones se aplican en la posición actual de lectura. Las mayúsculas y minúsculas se distinguen. El alfabeto de identificadores se limita a letras ASCII, dígitos y guion bajo. **KEYWORD.** Conjunto reservado: `print`, `int`, `return`, `if`, `else`. Se reconoce primero el identificador completo y luego se comprueba su pertenencia a este conjunto. Patrón para un lexema completo:

```
(?:print|int|return|if|else)
```

IDENTIFIER. Primera posición: letra o guion bajo; después se admiten también dígitos.

```
[A-Za-z_] [A-Za-z0-9_]*
```

CONSTANT. Enteros y decimales sin signo. Ambos se reportan bajo la misma categoría.

```
[0-9]+(?:\.[0-9]+)?
```

LITERAL. Cadenas con comillas rectas o tipográficas emparejadas. No se admiten escapes ni saltos de línea dentro de una cadena.

```
"[^"\\r\\n]*" | "[^'\\r\\n]*"
```

OPERATOR. Los operadores de dos caracteres se colocan antes que los de uno.

```
==|!=|<=|>=| [=+*/<>-]
```

PUNCTUATION. Paréntesis, llaves, punto y coma y coma.

```
[(){}; ,]
```

Uso **PUNCTUATION** como nombre de salida para coincidir con la tarea #10. Corresponde a la categoría de puntuación denominada **PUNCTUATOR** en el documento anterior. El Developer debe usar un solo nombre de forma consistente.

3. Ejemplos por categoría

Categoría	Ejemplos válidos	Ejemplos que no corresponden
KEYWORD	<code>int</code> , <code>print</code>	<code>intx</code> y <code>printf</code> : identificadores.
IDENTIFIER	<code>main</code> , <code>_x</code> , <code>if2</code>	<code>2a</code> : inicio numérico; <code>a-b</code> : tres tokens.
CONSTANT	<code>10</code> , <code>0</code> , <code>3.14</code>	<code>3.</code> , <code>123abc</code> , <code>1.2.3</code> : mal formados.
LITERAL	<code>"Hola"</code> , <code>""</code>	Cadena sin cerrar o con salto de línea.
OPERATOR	<code>=</code> , <code>>=</code> , <code>+</code>	<code>&</code> , <code>!</code> : no definidos.
PUNCTUATION	<code>(</code> , <code>;</code> , <code>}</code>	<code>[</code> , <code>]</code> : no definidos.

Que un ejemplo no pertenezca a una categoría no significa necesariamente que toda la entrada sea un error. Por ejemplo, `a-b` se divide en identificador, operador e identificador.

4. Prioridad y conflictos

Se conserva la coincidencia válida más larga desde la posición actual. Un identificador se lee completo antes de reclasificarlo: `if` es KEYWORD y `if2` es IDENTIFIER. Así se evita separar `intx` como `int` seguido de `x`. `printf` es IDENTIFIER porque no pertenece al conjunto reservado.

En operadores, `>=` y `==` son un solo token; se prueban antes que `>` y `=`. Si se usan patrones numéricos separados, el decimal se prueba antes que el entero. Los signos son operadores independientes: `-10` produce OPERATOR y CONSTANT.

Una cadena se reconoce como una unidad: los espacios y operadores que aparezcan dentro de ella no generan tokens adicionales. Esta versión no contempla comentarios.

5. Errores y recuperación

Antes de aceptar un prefijo numérico válido, se revisa el candidato completo: una secuencia que empieza con dígito y continúa con letras, guion bajo o puntos se consume hasta el siguiente separador u operador. Si no cumple el patrón numérico, se registra como un error completo. Por ejemplo, `123abc`, `3.` y `1.2.3` no se dividen en tokens parcialmente válidos.

Una cadena sin cerrar se consume hasta el salto de línea o fin de archivo y se reporta como error. Un carácter ilegal, como `@`, se registra y se avanza un carácter. Cada error conserva el lexema y su posición inicial. El análisis continúa y los errores no se suman al total de tokens válidos. Las líneas y columnas comienzan en 1; cada carácter, incluida la tabulación, avanza una columna, y el salto de línea reinicia la columna en 1.

6. Ejemplos del profesor

`int a = 10;` genera KEYWORD, IDENTIFIER, OPERATOR, CONSTANT y PUNCTUATION: **5 tokens**.

`printf("This is an example");` genera IDENTIFIER, PUNCTUATION, LITERAL, PUNCTUATION y PUNCTUATION: **5 tokens**. Ambas líneas juntas producen **10 tokens válidos**.

Los patrones de identificadores y constantes son la base del modelo AFND/AFD de la tarea #11. El enunciado menciona "Tokens: 40" sin precisar su interpretación; este punto debe confirmarse con el profesor.